

COURSE MATERIAL



Module 4: Constraints, Database Design & Normalization

Module 4 - 8 Hours

Constraints, Database Design & Normalization

Module 4 at a glance

Duration	8 hours (4 hours theory, 4 hours lab)
Prerequisites	Modules 1 to 3
You will build	A normalized replacement for the branch customer master, plus the ICICI Bank ER model
You will carry forward	The design reasoning behind every table you have been using since Module 1

Learning outcomes

By the end of this module you can:

1. Distinguish super, candidate, primary, alternate, composite, surrogate and foreign keys.
2. Explain referential integrity and choose the correct cascading rule for a banking table.
3. Draw an ER model in crow's foot notation and convert it into a physical schema.
4. Resolve a many-to-many relationship with a junction table.
5. Take a table through 1NF, 2NF, 3NF and BCNF, naming the anomaly each stage removes.
6. Decide when to denormalize deliberately, and how to keep denormalized data correct.

The business problem

Ticket ICICI-4577 - Retire the branch customer master spreadsheet

Every branch maintains `CUSTOMER_MASTER.xls`. It has been loaded into the database as one wide table and three reports now read it. Audit has raised two findings: the same branch name appears with three different spellings, and closing an account deleted the customer's phone number. Fix the design, and produce a proper ER model so this cannot happen again with the new credit card system.

1. Keys

1.1 The family of keys

Key	Definition	ICICI Bank example
Super key	Any set of columns that uniquely identifies a row	(<code>account_id</code> , <code>balance</code>)
Candidate key	A minimal super key: remove any column and uniqueness is lost	<code>account_id</code> ; separately, <code>account_number</code>
Primary key	The candidate key chosen as the row's identity	<code>account_id</code>
Alternate key	The candidate keys not chosen	<code>account_number</code>
Composite key	A key spanning more than one column	(<code>loan_id</code> , <code>installment_no</code>)
Foreign key	A column whose value must exist as a key in another table	<code>account.customer_id</code>

(`account_id`, `balance`) is a super key because it is unique, but it is not a candidate key: drop `balance` and it is still unique, so it was not minimal.

1.2 Choosing a primary key

Five criteria, in order of how often they are violated:

1. **Stable** - it must never need to change. This is the one people get wrong.
2. **Unique** - no two rows may ever share it.
3. **Not null** - every row must have one.
4. **Minimal** - as few columns as will do the job.
5. **Meaningless, preferably** - business meaning changes; identity must not.

A phone number is unique today. It is not stable, because people change numbers, and when they do, every foreign key referencing it has to change too. An email address, a PAN number, an account number and a customer's name all fail the stability test for the same reason.

1.3 Surrogate versus natural keys

	Surrogate	Natural
Example	<code>customer_id = 1001</code>	<code>pan_number = 'ABCDE1234F'</code>
Business meaning	None	Yes
Stability	Permanent by construction	Changes when the business does
Size in foreign keys	Small integer	Often a long string, repeated in every child row
Readability	Poor without a join	Good

ICICI Bank uses surrogate primary keys throughout and enforces the natural key separately:

```
customer_id INTEGER NOT NULL,           -- surrogate: the identity
pan_number  CHAR(10) NOT NULL UNIQUE,   -- natural: still guaranteed unique
CONSTRAINT pk_customer PRIMARY KEY (customer_id)
```

This gets both benefits. The identity never changes, and the business rule that no two customers share a PAN is still enforced by the engine.

2. Referential integrity

2.1 What it guarantees

A foreign key value must either match an existing primary key or be NULL. The engine checks this on insert, on update of the child, and on delete or key-update of the parent.

Without it you get **orphan rows**: an account belonging to no customer, a transaction against no account. Orphans are dangerous precisely because they are invisible. The row looks normal, joins silently drop it, and every total that should include it is quietly a little wrong.

2.2 Cascading rules

Rule	On delete of the parent	On update of the parent key
NO ACTION	Reject, checked at the end of the statement	Reject
RESTRICT	Reject immediately	Reject
CASCADE	Delete the child rows too	Update the child rows too
SET NULL	Set the child's foreign key to NULL	Same
SET DEFAULT	Set the child's foreign key to its declared default	Same

NO ACTION and RESTRICT differ only in timing. RESTRICT fires immediately; NO ACTION defers to the end of the statement, which matters when a deferred constraint or a trigger might fix the situation in between. In everyday use they behave identically.

2.3 Choosing the rule

Child table	Rule	Reason
account referencing customer	RESTRICT	Deleting a customer must never silently remove their accounts
account_txn referencing account	RESTRICT	Transaction history is regulated evidence
loan_payment referencing loan	CASCADE	An instalment schedule has no meaning without its loan
beneficiary referencing customer	CASCADE	A payee list is a preference, not a financial record
employee.manager_id referencing employee	SET NULL	A manager leaves; their team must not be deleted with them

Default to RESTRICT on anything financial. CASCADE is correct only when the child is genuinely owned by the parent and is meaningless without it. The test to apply: if the parent disappeared, would anyone ever need this child row again? For a loan's instalment schedule, no. For a customer's transactions, absolutely yes.

2.4 The second audit finding, explained

"Closing an account deleted the customer's phone number."

In the flat table, the customer's details and the account's details occupy **the same row**. There is nowhere else the phone number is stored, so removing the account row removes the customer too.

This is a **deletion anomaly**, and no constraint can fix it. `ON DELETE RESTRICT` would not help, because there is no parent-child relationship to restrict - the two entities were merged into one row by the design. Only decomposition fixes it, which is what normalization is.

3. Entity relationship modelling

3.1 The building blocks

Concept	Definition	Becomes
Entity	A thing the bank must remember	A table
Attribute	A fact about an entity	A column
Relationship	How two entities connect	A foreign key, or a junction table

A practical heuristic when reading a requirements document: nouns become entities, facts about those nouns become attributes, and verbs become relationships. "A **customer** holds many **accounts**, each opened at a **branch**" gives you three entities and two relationships in one sentence.

3.2 Cardinality and optionality

Cardinality is how many of one side relate to the other.

Cardinality	ICICI Bank example	Physical implementation
One to one	An account and its passbook record	Foreign key with a UNIQUE constraint, or merge the tables
One to many	One customer holds many accounts	Foreign key on the many side
Many to many	Customers hold many products, products are held by many customers	A junction table

Optionality is whether the relationship is compulsory.

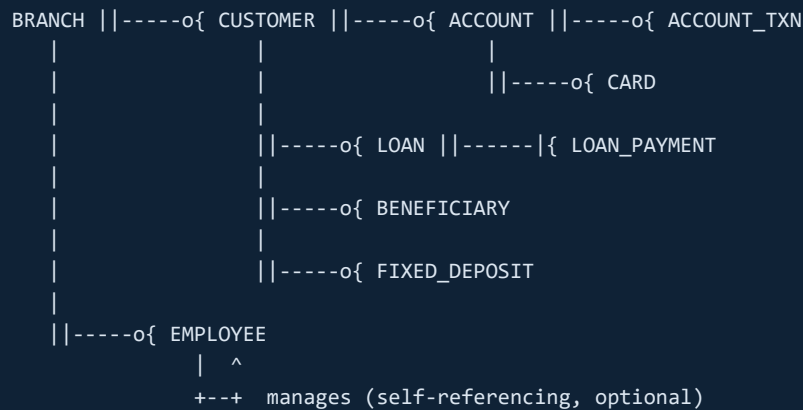
```
CUSTOMER ||-----o{ ACCOUNT
```

```
|| on the customer side : an account must have exactly one customer
o{ on the account side  : a customer may have zero or many accounts
```

Crow's foot symbol	Meaning		
`		`	Exactly one, mandatory
`o	`		Zero or one, optional
`}	`		One or many, mandatory
o{	Zero or many, optional		

Optionality decides nullability. Because an account must have a customer, `account.customer_id` is **NOT NULL**. Because an employee may have no manager, `employee.manager_id` is nullable.

3.3 The ICICI Bank ER model



Nine entities and ten relationships. Every join written in Module 3 traverses one of these lines. Note **LOAN** | |-----|{ **LOAN_PAYMENT**: a loan must have at least one payment row in its schedule, which is why that side is mandatory.

3.4 Resolving many to many

A many-to-many relationship cannot be stored directly, because there is no side to put the foreign key on. It always resolves into two one-to-many relationships around a **junction table**.

WRONG	RIGHT
CUSTOMER }o-----o{ PRODUCT	CUSTOMER --o{ CUSTOMER_PRODUCT }o-- PRODUCT

```
CREATE TABLE customer_product (
    customer_id INTEGER NOT NULL,
    product_code VARCHAR(10) NOT NULL,
    enrolled_on DATE NOT NULL,
    status VARCHAR(10) NOT NULL,
    CONSTRAINT pk_customer_product PRIMARY KEY (customer_id, product_code),
    CONSTRAINT fk_cp_customer FOREIGN KEY (customer_id) REFERENCES customer(customer_id),
    CONSTRAINT fk_cp_product FOREIGN KEY (product_code) REFERENCES product(product_code)
);
```

The junction table's primary key is the pair of foreign keys, which automatically prevents enrolling the same customer in the same product twice. It can also carry attributes of its own - here, when the customer enrolled and whether the enrolment is still active. Those attributes belong to the **relationship**, not to either entity, and there is nowhere else to put them.

The `products` column holding 'FD,LOAN,CARD' in the flat table is a many-to-many relationship that was never resolved.

3.5 Weak entities

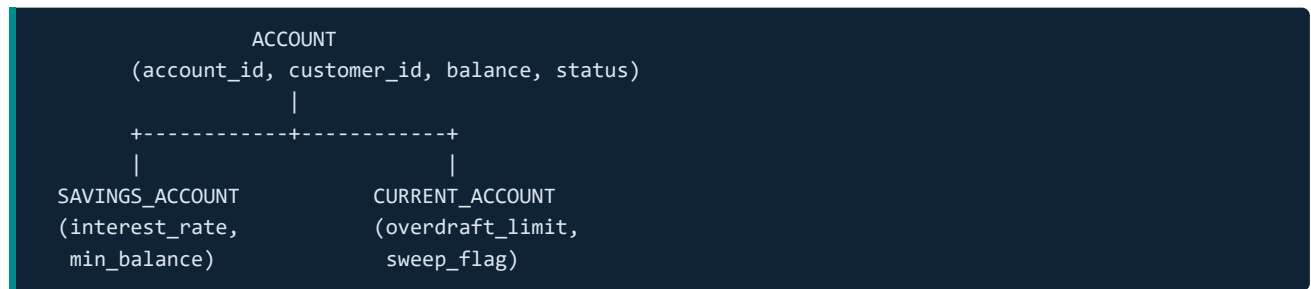
A **weak entity** cannot exist independently of its parent and is identified partly by the parent's key.

```
CONSTRAINT pk_loan_payment PRIMARY KEY (loan_id, installment_no),
CONSTRAINT fk_lp_loan FOREIGN KEY (loan_id)
    REFERENCES loan(loan_id) ON DELETE CASCADE
```

Instalment number 7 means nothing on its own; it is instalment 7 **of loan 5003**. Weak entities are exactly where **ON DELETE CASCADE** is the right choice.

3.6 Generalization

When several entities share most attributes but differ in a few, you have a choice.



Approach	Choose when
Separate subtype tables	Subtypes differ substantially and are usually queried separately
One table with a type discriminator and nullable columns	Subtypes differ slightly and are usually queried together

ICICI Bank uses the second: one `account` table with an `account_type` column. That is why `interest_rate` is nullable - it is meaningless on a current account. The trade-off is honest: fewer joins, at the cost of columns that do not apply to every row.

3.7 From ER model to physical tables

1. Every entity becomes a table.
2. Every attribute becomes a column, with a type and a nullability decision.
3. Choose a primary key for each table, preferring a surrogate.
4. Every one-to-many relationship becomes a foreign key on the many side.
5. Every many-to-many becomes a junction table with a composite primary key.
6. Set a cascading rule on every foreign key, deliberately.
7. Add `CHECK` constraints for every business rule that can be expressed on a single row.

Steps 6 and 7 are the ones that get skipped under deadline pressure, and they are the two the auditor asks about.

4. Normalization

4.1 The three anomalies

Normalization exists to remove three specific failures.

Anomaly	What happens	In the flat table
Insert	A fact cannot be recorded without an unrelated one	A new branch cannot be registered until it has its first customer
Update	The same fact must be changed in many rows	Renaming a branch means updating every customer row, and missing one produces divergent spellings
Delete	Removing one fact destroys another	Closing an account removes the customer's phone number

Both findings on ticket ICICI-4577 are on this list.

4.2 Functional dependency

A → B means: given a value of A, there is exactly one corresponding value of B.

```
customer_id -> first_name, last_name, phone, city
branch_code -> branch_name, branch_ifsc, branch_city
account_no  -> account_type, balance
```

A **determinant** is the left-hand side. Normalization is the process of ensuring every non-key column depends on the key of the table it sits in, and on nothing else. The traditional summary is worth memorising:

Every non-key attribute must depend on **the key, the whole key, and nothing but the key.**

The three clauses correspond exactly to 1NF, 2NF and 3NF.

4.3 First Normal Form

Rules: every value is atomic, there are no repeating groups, and every row is uniquely identifiable.

```
VIOLATION 1: products = 'FD,LOAN,CARD'          -- not atomic
VIOLATION 2: acct_no_1, acct_no_2, acct_no_3    -- repeating group
```

A comma-separated list in a column cannot be indexed, joined, constrained or counted correctly. Finding customers holding a loan means `LIKE '%LOAN%'`, which also matches a hypothetical `HOMELOAN`, cannot use an index, and gives you no way to record when the customer took the product.

A repeating group is worse, because it fixes an arbitrary limit in the schema. A customer with three accounts needs an `ALTER TABLE`, and every query that reads accounts has to be rewritten to look at the new column.

The fix turns columns into rows:

```
BEFORE
cust_id cust_name  acct_no_1 bal_1  acct_no_2 bal_2  products
1005    K. Gupta   ...9005   91729   ...9006   27637   FD, LOAN, CARD
```

```

AFTER
customer          customer_account      customer_product
cust_id cust_name  cust_id acct_no  balance  cust_id product
1005    K. Gupta   1005    ...9005    91729   1005    FD
                                1005    ...9006    27637   1005    LOAN
                                                1005    CARD

```

A customer may now hold any number of accounts and any number of products, without a schema change.

4.4 Second Normal Form

Rules: already in 1NF, and no non-key column depends on only part of a composite primary key.

```

ACCOUNT_TXN_FLAT  PRIMARY KEY (account_id, txn_id)

account_id, txn_id -> amount, txn_date    depends on the whole key    OK
account_id         -> account_type, balance depends on PART of the key  2NF violation

```

`account_type` is repeated on every one of that account's transactions. Change the account type and you must update thousands of transaction rows, or they disagree.

2NF only applies to tables with a composite primary key. A single-column key cannot have a partial dependency, so any 1NF table with a simple key is automatically in 2NF. This is one reason surrogate keys make schemas easier to reason about.

The fix moves the partially dependent columns to a table keyed on the part they depend on:

```

ACCOUNT          ACCOUNT_TXN
account_id  PK   txn_id      PK
account_type                account_id  FK
balance                                amount
                                txn_date

```

4.5 Third Normal Form

Rules: already in 2NF, and no non-key column depends on another non-key column.

```

CUSTOMER_FLAT  PRIMARY KEY (customer_id)

customer_id -> branch_code    depends on the key    OK
branch_code -> branch_name    depends on a NON-key column  transitive, 3NF violation

```

`branch_name` is a fact about the **branch**, not about the customer. It has ended up in the customer table because the branch is mentioned there, and it is therefore repeated once per customer of that branch.

This is precisely the first audit finding. In the flat table, branch BR03 appears with four different spellings - `Koramangala`, `Koramangla`, `Kormangala` and `Koramangala` with a trailing space - because four different people typed it on four different days and nothing forced them to agree.

The fix moves the transitively dependent columns into their own table:

```

CUSTOMER                                BRANCH
customer_id  PK                        branch_id  PK
first_name                                     branch_name
last_name                                       ifsc_code
branch_id    FK                            city

```

The branch name is now stored once. It **cannot** have four spellings, because there is only one place it exists. Renaming a branch is a single-row update.

This is the schema you have been using since Module 1.

4.6 Boyce-Codd Normal Form

Rule: every determinant must be a candidate key. BCNF is a stricter form of 3NF, and the two differ only when a table has overlapping candidate keys.

```

BRANCH_RM_ASSIGNMENT (branch_id, product_code, rm_id)

(branch_id, product_code) -> rm_id      candidate key      OK
rm_id                    -> product_code  rm_id determines a
                                         column but is NOT a
                                         candidate key      BCNF violation

```

The business rule behind this is that each relationship manager handles exactly one product. Storing that fact in the assignment table repeats it on every branch the manager covers.

The fix splits the table:

```

RM_PRODUCT          BRANCH_RM
rm_id      PK      branch_id  PK
product_code          rm_id    PK

```

Most 3NF tables are already in BCNF. When one is not, it is nearly always this shape: a table with two overlapping candidate keys and a dependency running from part of one to part of the other.

4.7 Higher normal forms

Form	Removes	In practice
4NF	Multi-valued dependencies - two independent one-to-many facts in one table	Occasionally relevant
5NF	Join dependencies	Rare
6NF	Every non-key column in its own table	Temporal data warehouses only

3NF, or BCNF where it differs, is the target for an OLTP banking schema. Beyond that, the extra joins usually cost more than the anomalies they prevent.

4.8 The whole journey

Form	Rule	Anomaly removed
1NF	Atomic values, no repeating groups	Cannot store, index or query multiple values properly

Form	Rule	Anomaly removed
2NF	No partial dependency on a composite key	Parent facts repeated on every child row
3NF	No transitive dependency	Update anomaly, divergent spellings, orphaned lookups
BCNF	Every determinant is a candidate key	Residual redundancy with overlapping candidate keys

5. Functions in the schema

Modules 2 and 3 used functions to shape what a query returns. Here they do a different job: they enforce the design. A rule written into the schema is met by every application, every batch job and every DBA at a prompt. A rule written in application code is met by whoever remembered it.

This is Codd's rule 10 from Module 1, made concrete.

5.1 DEFAULT expressions

A `DEFAULT` runs when the `INSERT` does not supply the column.

```
CREATE TABLE ex4_locker_agreement (
  agreement_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  customer_id INTEGER NOT NULL REFERENCES customer(customer_id),
  start_date DATE NOT NULL DEFAULT CURRENT_DATE,
  status VARCHAR(12) NOT NULL DEFAULT 'ACTIVE',
  created_at TIMESTAMP NOT NULL DEFAULT NOW(),
  reference UUID NOT NULL DEFAULT gen_random_uuid()
);
```

Default	Evaluated	Use
A literal such as <code>'ACTIVE'</code>	Once, at insert	The normal opening state
<code>CURRENT_DATE</code> , <code>NOW()</code>	At each insert	When a row was created
<code>gen_random_uuid()</code>	At each insert	An identifier safe to expose outside
<code>GENERATED ALWAYS AS IDENTITY</code>	At each insert	The standard surrogate key

`GENERATED ALWAYS AS IDENTITY` is the standard form and replaces `SERIAL`. `SERIAL` is a PostgreSQL shorthand that creates a sequence behind your back and leaves the column an ordinary integer, so nothing stops an application supplying its own value. `IDENTITY` refuses.

A default is not a constraint. `DEFAULT 'ACTIVE'` fills a blank. It does not stop someone inserting `'BANANA'`. That needs the `CHECK` in 5.2.

5.2 Functions inside CHECK constraints

A `CHECK` may use functions, and this is where most real validation lives.

```
ALTER TABLE branch
  ADD CONSTRAINT ck_branch_ifsc
  CHECK (ifsc_code ~ '^[A-Z]{4}0[A-Z0-9]{6}$');
```

`~` is the PostgreSQL regular expression match operator. That one line rejects every malformed IFSC at the door, from every application at once. ICICI Bank's own codes such as `ICIC0000101` pass: four letters, a zero, then six characters.

The validations a retail bank actually needs:

Rule	Constraint
IFSC format	<code>ifsc_code ~ '^[A-Z]{4}0[A-Z0-9]{6}\$'</code>
PAN format	<code>pan ~ '^[A-Z]{5}[0-9]{4}[A-Z]\$'</code>
Indian mobile	<code>phone ~ '^[6-9][0-9]{9}\$'</code>
An email has one at sign	<code>POSITION('@' IN email) > 1</code>
No leading or trailing space	<code>branch_name = TRIM(branch_name)</code>
A balance is never negative	<code>balance >= 0</code>
An end date follows a start date	<code>end_date > start_date</code>

That fifth rule is worth pausing on. `CHECK (branch_name = TRIM(branch_name))` would have made 'Koramangala ' impossible, and three quarters of the cleanup in this module's lab would never have been needed.

The rule a CHECK may not express

A **CHECK** may depend only on the values in its own row. It may not run a subquery, may not look at another table, and should not depend on the clock.

```
-- Wrong, even where the database lets you write it
CHECK (start_date >= CURRENT_DATE)
```

A constraint is re-evaluated when a backup is restored. A row that was valid the day it was inserted fails the check a year later, and the restore stops part way through. Any rule involving now belongs in a trigger or in the application, not in a **CHECK**.

Likewise, "the customer must exist" is not a **CHECK**. It is a **FOREIGN KEY**, which is section 2.

5.3 Generated columns

A generated column is computed by the database from other columns in the same row, and stored. It cannot be written to.

```
ALTER TABLE account
  ADD COLUMN masked_number TEXT
  GENERATED ALWAYS AS ('XXXXXXXXX' || RIGHT(account_number, 4)) STORED;
```

Every application now sees the same masking, computed the same way, with no chance of one team formatting it differently. Compare that with a view, which computes on each read and cannot be indexed directly.

Approach	Computed	Can be indexed	Can go stale
Generated column	On write	Yes	No
View column	On read	Not directly	No
An ordinary column the application fills	Never	Yes	Yes

The expression must be immutable. It may use only the row's own columns and functions whose output depends on nothing else, so `CURRENT_DATE` and `gen_random_uuid()` are both refused. That restriction is what

guarantees the stored value can never disagree with the expression.

This is the honest version of denormalization, which is section 6. The database owns the derived value, so it cannot drift.

5.4 The functions normalization needs

Section 4 decomposed a flat table. Doing that against real data means cleaning it first, and four functions do most of the work.

Function	Fixes
TRIM(s)	'Koramangala ' and 'Koramangala' becoming one value
UPPER(s) or LOWER(s)	'koramangala' and 'Koramangala' becoming one value
REGEXP_REPLACE(s, p, r, 'g')	Stray punctuation, doubled spaces
NULLIF(s, '')	An empty string that should have been NULL

```
SELECT DISTINCT branch_name AS raw,
       UPPER(REGEXP_REPLACE(TRIM(branch_name), '\s+', ' ', 'g')) AS normalised
FROM   branch_customer_flat
WHERE  branch_code = 'BR03'
ORDER BY raw;
```

raw	normalised
Koramangala	KORAMANGALA
Koramangala	KORAMANGALA
Koramangla	KORAMANGLA
Kormangala	KORMANGALA

Read that output carefully. Trimming and upper-casing collapses four spellings into three, not one. `Koramangla` and `Kormangala` are misspellings, not formatting, and no function can decide which is correct. That decision needs the RBI IFSC directory and a human, which is the exercise question in this module.

****Normalize formatting automatically. Never normalize meaning automatically.**** Whitespace and case are formatting. A misspelling is a competing claim about a fact, and resolving it silently is how a cleanup enshrines an error with no record that a decision was ever made.

5.5 Where to put a rule

The rule	Where it belongs
A value must be one of a fixed set	CHECK, or a lookup table with a foreign key
A value must match a format	CHECK with a regular expression
A value must exist in another table	FOREIGN KEY
A value must be unique	UNIQUE

The rule	Where it belongs
A column is derived from others in the row	Generated column
A rule involving today's date	A trigger, or the application
A rule spanning several rows or tables	A trigger, or a procedure

Everything in the top half of that table is enforced by the database for every caller. Everything in the bottom half depends on code that someone can forget to call, which is why the audit findings in Module 9 are mostly about the bottom half.

6. Denormalization

6.1 When to break the rules deliberately

Normalization optimises for correct, cheap **writes**. Reporting wants fast **reads**. Denormalization trades the first for the second, and it is a legitimate engineering decision when three conditions hold: you can name the query that needs it, you have measured the problem, and you have a control that keeps the duplicated data honest.

Situation	Denormalization	Covered in
Mini-statement recomputes a balance from 40 million rows	Store <code>account.balance</code> and maintain it	This module
The warehouse joins eight tables for every report	Star schema with a wide fact table	Module 10
A screen shows the branch name on every row	Duplicate <code>branch_name</code> into the summary table	Module 10
The morning pack takes 40 seconds	Overnight summary table	Modules 10 and 11

6.2 ICICI Bank's own deliberate denormalization

`account.balance` is derivable. It is stored anyway.

```
-- Correct, and reads every transaction the account has ever had
SELECT SUM(CASE WHEN txn_type = 'CREDIT' THEN amount ELSE -amount END)
FROM   account_txn
WHERE  account_id = 9003;
```

On an account with twenty years of history that is unusable for a balance enquiry that must answer in milliseconds. So the bank stores the balance and updates it in the **same transaction** as the posting:

```
BEGIN;
  INSERT INTO account_txn (...) VALUES (...);
  UPDATE account SET balance = balance - 5000 WHERE account_id = 9003;
COMMIT;
```

Atomicity guarantees the two never diverge because of a crash. What it does not protect against is a code path that inserts a transaction and forgets the update.

6.3 Keeping denormalized data honest

Technique	How it works
Same transaction	Update the derived value inside the same <code>BEGIN...COMMIT</code> as the source change
Trigger	The database maintains it, so no application can forget - Module 10
Scheduled recomputation	Rebuild the summary nightly - Module 10
Reconciliation report	Compare stored against computed and alert on any difference

The reconciliation is not optional:

```
SELECT a.account_id,  
       a.balance AS stored_balance,  
       COALESCE(SUM(CASE WHEN t.txn_type = 'CREDIT' THEN t.amount ELSE -t.amount END), 0)  
         AS computed_balance  
FROM   account a  
       LEFT JOIN account_txn t ON t.account_id = a.account_id  
GROUP BY a.account_id, a.balance  
HAVING a.balance <> COALESCE(SUM(CASE WHEN t.txn_type = 'CREDIT'  
                                THEN t.amount ELSE -t.amount END), 0);
```

Every denormalization needs a reconciliation that proves it is still true. A derived value with no control is a value nobody can trust, and in a bank that means it will eventually be wrong in a way that reaches a customer.

6.4 Two schemas, two targets

	OLTP core banking	Reporting warehouse
Target normal form	3NF or BCNF	Star schema, deliberately denormalized
Optimised for	Correct, fast writes	Fast aggregate reads
Redundancy	Eliminated	Accepted and controlled
Update frequency	Continuous	Nightly load
Who guarantees correctness	Declared constraints	The ETL job

Both exist at ICICI Bank. Module 10 builds the second, and Module 11 moves data between them.

7. Closing the ticket

Audit finding	Root cause	Fixed by
Four spellings of one branch name	Transitive dependency: <code>customer_id -> branch_code -> branch_name</code>	3NF - branch details moved into <code>branch</code>
Closing an account deleted the phone number	Two entities merged into one row	1NF and 3NF decomposition, plus <code>ON DELETE RESTRICT</code>

The delivered design is the nine-table schema this programme has used since Module 1: `branch`, `customer`, `account`, `account_txn`, `loan`, `loan_payment`, `card`, `beneficiary` and `employee`, with `fixed_deposit` added in Module 2.

Terms introduced in this module

Term	Meaning
Super key, candidate key, alternate key	Unique column sets: any, minimal, and the ones not chosen as primary
Surrogate key	A meaningless system-generated identifier
Natural key	A business value that is unique
Referential action	<code>RESTRICT</code> , <code>CASCADE</code> , <code>SET NULL</code> , <code>SET DEFAULT</code> , <code>NO ACTION</code>
Orphan row	A child row whose parent no longer exists
Cardinality, optionality	How many, and whether compulsory
Junction table	Resolves a many-to-many into two one-to-many relationships
Weak entity	Cannot exist without its parent, identified partly by the parent's key
Functional dependency, determinant	<code>A -> B</code> , and the left-hand side of it
Insert, update, delete anomaly	The three failures normalization removes
1NF, 2NF, 3NF, BCNF	The key, the whole key, nothing but the key, and every determinant a candidate key
Denormalization	Deliberate, measured, controlled redundancy for read performance

Where this module leads

You learned	It becomes
Normalized schemas need joins	Module 5, making those joins fast with indexes
<code>account.balance</code> is denormalized	Module 5's stored procedures, and Module 6's transactions
Reconciliation of derived values	Module 9 monitoring, and Module 11 migration validation
Star schemas and deliberate redundancy	Module 10, the SQL Server reporting warehouse
The ER model	Module 12, where you design the capstone schema yourself